

随机梯度法与Mini Batch

[后续第5讲_随机梯度与MiniBatch.pdf](#)

本章主线：全梯度太贵 → 随机梯度替代全梯度 → Mini-batch 折中 → 分析随机噪声 → 学习率与 batch size 调参 → Momentum/Adam 改进 SGD

了解几种下降法：

很多经验风险最小化问题都可以写成：

$$f(w) = \frac{1}{n} \sum_{i=1}^n f_i(w).$$

例如，线性回归中，单样本损失可以写成 $f_i(w) = \frac{1}{2}(x_i^T w - y_i)^2$ ；分类模型中， $f_i(w)$ 可以是第 i 个样本的交叉熵损失。这里的共同点是：**总目标函数不是凭空出现的，而是所有样本损失的平均**。因此，优化模型参数 w 的本质就是：让模型在所有训练样本上的**平均损失**尽可能小。

普通梯度下降 GD：

GD，即 Gradient Descent，使用完整梯度更新：

$$w^{k+1} = w^k - \alpha_k \left(\frac{1}{n} \sum_{i=1}^n \nabla f_i(w^k) \right).$$

其中， α_k 是第 k 步的学习率，也就是前面课程中常说的步长。梯度下降的优点是**方向准确**，因为它使用了所有样本的平均梯度；缺点是**单步代价高**，因为每更新一次都要遍历全部训练样本。当**样本量很大**时，GD 的**更新频率会很低**。

随机梯度下降 SGD：

SGD，即 Stochastic Gradient Descent，每次**只随机抽取一个样本 i_k** ，用该样本的梯度近似完整梯度：

$$w^{k+1} = w^k - \alpha_k \nabla f_{i_k}(w^k).$$

SGD 的优点是单步代价极低，因为每次**只需要计算一个样本的梯度**；缺点是方向**噪声**很大，因为一个样本并不能完全代表整体数据分布。

Mini-batch：使用一小批样本的平均梯度

Mini-batch 方法**介于 GD 和 SGD 之间**。第 k 步**随机抽取一小批样本 B_k** ，然后计算这一批样本的**平均梯度**：

$$w^{k+1} = w^k - \alpha_k \left(\frac{1}{|B_k|} \sum_{i \in B_k} \nabla f_i(w^k) \right).$$

这里 B_k 是第 k 次更新抽到的样本集合， $|B_k|$ 是这个集合里的样本数量，也就是 batch size。

总结：

GD：使用全部样本

$$w^{k+1} = w^k - \alpha_k \left(\frac{1}{n} \sum_{i=1}^n \nabla f_i(w^k) \right)$$

方向准，但每一步都要遍历全部数据。

SGD：使用一个样本

$$w^{k+1} = w^k - \alpha_k \nabla f_{i_k}(w^k)$$

每步便宜，但方向噪声大。

Mini-batch：使用一小批样本

$$w^{k+1} = w^k - \alpha_k \left(\frac{1}{|B_k|} \sum_{i \in B_k} \nabla f_i(w^k) \right)$$

在计算效率和梯度稳定性之间折中；现代深度学习最常用。

方法	单步代价	训练直觉
GD	需要计算全部 n 个样本的梯度	一步方向准，但更新频率低
SGD	只计算 1 个样本的梯度	一步方向噪声大，但可以很快更新很多步
Mini-batch	计算一小批样本的平均梯度	借助并行计算，在速度和稳定性之间折中

小结

当全梯度的单步成本成为主要瓶颈时，我们可以接受“有噪声但便宜”的方向，用更多次快速更新换取更好的单位时间进展。

机器学习理解：

机器学习需要大量样本，本质上是在最小化所有样本上的平均损失：

$$\min_w \frac{1}{n} \sum_{i=1}^n f_i(w).$$

模型不是为了在某一张图片上表现好，而是要在大量样本的平均意义上表现好。原始图片是 8×8 的灰度矩阵。矩阵中每个位置是一个像素值，表示该位置的灰度深浅。但本案例先使用线性模型，而线性模型通常处理的是向量，所以需要把图片矩阵拉平成向量：

$$x_i = (p_{i,1}, p_{i,2}, \dots, p_{i,64})^T \in \mathbb{R}^{64}.$$

8×8 图片 $\Rightarrow$ 64 维特征向量.

接着对特征做标准化，即减均值、除标准差。标准化的目的不是改变任务本身，而是让不同像素维度的数值尺度更接近，避免某些像素维度因为数值范围较大而主导梯度。

在二分类任务中，模型要输出“这张图片是数字 5 的概率”。先用线性模型计算一个打分：

$$w^T x_i.$$

然后用 sigmoid 函数把这个打分变成 0 到 1 之间的概率：

$$p_i(w) = \sigma(w^T x_i) = \frac{1}{1 + e^{-w^T x_i}}.$$

如果 $p_i(w)$ 越接近 1，模型越认为这张图是 5；如果 $p_i(w)$ 越接近 0，模型越认为这张图不是 5。真实标签为 $\tilde{y}_i \in \{0, 1\}$ ，预测概率为 $p_i(w)$ 。讲义使用交叉熵损失：

$$f_i(w) = - [\tilde{y}_i \log p_i(w) + (1 - \tilde{y}_i) \log(1 - p_i(w))].$$

- 如果使用 GD，那么每一步要计算全梯度： $\nabla f(w) = \frac{1}{n} \sum_{i=1}^n \nabla f_i(w)$
- 如果使用 SGD，那么每一步只随机抽一张图片 i_k ，计算： $g_k = \nabla f_{i_k}(w^k)$
- 如果使用 Mini-batch，那么每一步抽一批图片 B_k ，计算： $g_{B_k}(w^k) = \frac{1}{|B_k|} \sum_{i \in B_k} \nabla f_i(w^k)$

方法	每步用多少张图	单步成本	实验中常见现象
GD	$n \approx 1797$	高	loss 曲线较平滑，但迭代少、耗时长
SGD	1	低	更新很快，曲线明显抖动
Mini-batch	32 等	中	折中：比 SGD 稳，比 GD 快

SGD 本质与原理剖析：

普通的梯度下降会把**所有样本的梯度**都求出来，然后用所有样本的梯度来求平均梯度作为方向，但 SGD 每一步只随机抽一个样本 i_k ，然后用这个样本的梯度作为更新方向：

$$g_k = \nabla f_{i_k}(w^k),$$

更新式为： $w^{k+1} = w^k - \alpha_k g_k$ 。

为啥可以保证能下降呢？

结论

在任意当前参数 w 处，随机梯度 g_k 是全梯度 $\nabla f(w)$ 的无偏估计：平均而言，它指向与 GD 相同的下降方向。

(os:数理统计知识发力了.....)

但是无偏只能指期望等于梯度，但是不代表每次抽出来的都是下降方向，可能会有某几次不仅不在下降方向，而且增加了函数值，但是从长远的来看，走很多步后噪声会相互抵消，平均轨迹仍然朝着梯度方向前进。

直观类比

GD：每步都看完整张地图；SGD：每步只问一个人指路，单次可能偏，但长期平均仍朝目标走。

在适当条件和**递减步长**下，**凸问题**中 SGD 可以得到类似

$$\mathbb{E}[f(\bar{w}^k) - f^*] = O\left(\frac{1}{\sqrt{k}}\right)$$

Mini-batch 无偏性：

Mini-batch 的梯度定义为：

$$g_{B_k}(w) = \frac{1}{|B_k|} \sum_{i \in B_k} \nabla f_i(w)$$

其中， B_k 是第 k 步抽到的一小批样本， $|B_k|$ 是 batch size。如果 B_k 中的样本也是随机抽取的，那么 Mini-batch 梯度的期望仍然等于全梯度：

$$\mathbb{E}[g_{B_k}(w)] = \nabla f(w)$$

Mini-batch 和 SGD 在“平均方向”上是一样的：它们都是全梯度的无偏估计。区别不在于是否无偏，而在于**方差大小**。

GD：问全班

方向最稳，但每步都要汇总全部意见，成本高。

SGD：随机问一人

每步很快；被问到的人若意见偏激，这一步就会偏，loss 曲线会抖。

噪声分析：

我们可以把随机梯度拆成：

$$g_k = \nabla f(w^k) + \varepsilon_k$$

推导过程：

对 $\varepsilon_k = g_k - \nabla f(w^k)$ 两边取条件期望：

$$\mathbb{E}[\varepsilon_k \mid w^k] = \mathbb{E}[g_k - \nabla f(w^k) \mid w^k]$$

利用期望的线性性：

$$\mathbb{E}[\varepsilon_k \mid w^k] = \mathbb{E}[g_k \mid w^k] - \mathbb{E}[\nabla f(w^k) \mid w^k]$$

一旦 w^k 固定，那么 $\nabla f(w^k)$ 也固定了。它不是随机变量了，而是一个确定向量。所以：

$$\mathbb{E}[\nabla f(w^k) \mid w^k] = \nabla f(w^k)$$

前面讲过随机梯度的无偏性：

$$\mathbb{E}[g_k \mid w^k] = \nabla f(w^k).$$

这句话的意思是：固定当前点 w^k 后，随机抽样得到的梯度 g_k 平均起来等于全梯度。所以我们就推出了噪声的期望为 0。

符号	含义
g_k	第 k 步实际用来更新的随机梯度
$\nabla f(w^k)$	当前点处真正想要的全梯度
ε_k	抽样带来的误差，也就是随机噪声

噪声的均值是 0（注意不是噪声为 0 哦~）。正负误差不会长期偏向一边，所以整体仍能下降。

情况	含义
$\varepsilon_k = 0$	这一单步随机梯度刚好等于全梯度，几乎不现实

情况	含义
$\mathbb{E}[\varepsilon_k] = 0$	很多次随机抽样的误差平均后为 0，这是 SGD 可行的关键

好处	代价
▶ 便宜：每步只算一个或少量样本的梯度； ▶ 可扩展：数据可流式到来，不必每次都扫全库； ▶ 随机性：非凸问题中有时帮助跳出差的驻点或平坦区。	▶ 抖动：单步方向方差大，loss 曲线不如 GD 平滑； ▶ 调参难：学习率、batch size 对稳定性更敏感； ▶ 不单调：可能出现某一步 loss 暂时上升。

知识点	结论
随机梯度拆解	$g_k = \nabla f(w^k) + \varepsilon_k$
噪声均值	$\mathbb{E}[\varepsilon_k w^k] = 0$
曲线抖动来源	随机噪声 ε_k 的波动
batch size 作用	batch 越大，噪声通常越小，曲线越平滑
SGD 的好处	便宜、可扩展、适合大规模和在线数据
SGD 的代价	抖动、不单调、对学习率和 batch size 敏感

Batch size 对训练的影响：

我们回顾一下 Mini batch，对梯度取平均：

$$g_{B_k}(w) = \frac{1}{|B_k|} \sum_{i \in B_k} \nabla f_i(w).$$

概念	含义
iteration	参数更新一次（每抽一个 batch 并更新，计 1 次）
batch B_k	第 k 次更新所用的样本集合
batch size $ B_k $	该批样本个数
epoch	训练集 n 个样本各被用到约一遍

batch size	梯度噪声	每个 epoch 的更新次数
小（如 1）	大，loss 曲线抖	多
中（如 32）	中等，常用	中等
大（如 512）	小，更平滑	少

SGD 的收敛性分析：

在凸、光滑、随机梯度方差有界，并且使用递减步长的情况下，SGD 可以得到典型的期望收敛率：

$$\mathbb{E}[f(\bar{w}^k) - f(w^*)] \leq O\left(\frac{1}{\sqrt{k}}\right).$$

这个地方我们想要证明的就是，长期来看，SGD 能够接近一阶最优条件达到的效果，而一阶最优条件就是 $\nabla f(w) = 0$ ，所以我们自然要研究 $\|\nabla f(w^k)\|^2$ 。这里 $\bar{w}^k$ 是前 k 步参数的平均。重点是理解：噪声项会以 α_k^2 的形式出现，所以递减步长可以逐渐压住噪声。具体推导过程见ppt吧，其实写的很详细了，整体思路就是：

光滑性 $\Rightarrow$ 单步下降不等式 $\Rightarrow$ 取条件期望 $\Rightarrow$ 用无偏性得到真梯度下降项 $\Rightarrow$ 用方差有界控制噪声 $\Rightarrow$ 累加 $\Rightarrow$ 望远镜求和 $\Rightarrow$ 得到平均意义下的收敛界。

最后得到类似：

$$\frac{\sum_{k=1}^K \alpha_k \mathbb{E} \|\nabla f(w^k)\|^2}{\sum_{k=1}^K \alpha_k} \leq \frac{f(w^1) - f^*}{c_1 \sum_{k=1}^K \alpha_k} + \frac{c_2 \sigma^2 \sum_{k=1}^K \alpha_k^2}{c_1 \sum_{k=1}^K \alpha_k}.$$

直接看结构：

平均梯度范数 $\leq$ 初始差距项 + 噪声累计项。

第一部分表示初始离最优点有多远。随着步数增加，累计步长 $\sum \alpha_k$ 变大，这个初始差距会被摊薄。

第二部分表示随机噪声带来的影响。这里 σ^2 是噪声强度， α_k^2 说明学习率越大，噪声影响越大。这也是要选择递减步长的原因，靠累计步长推动下降，同时靠递减学习率压制噪声。

SGD 非常依赖学习率（或者说步长）：

更新式 $w^{k+1} = w^k - \alpha_k g_k$ 中， α_k 同时影响两件事：

- ▶ 下降幅度：太小则进步慢，太大则可能越过最优点甚至发散；
- ▶ 噪声放大：项 $\frac{L\alpha_k^2}{2} \|g_k\|^2$ 表明， α_k 过大时随机噪声被放大，loss 更易抖动。

其他的修正方法：

Momentum / Nesterov：解决方向抖动

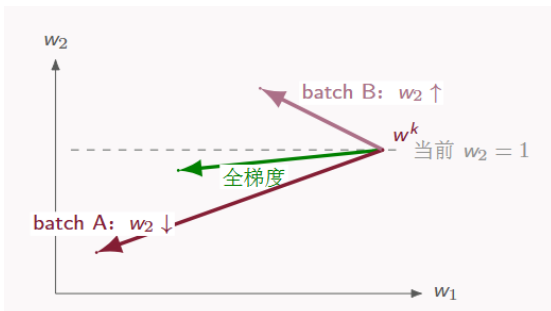
Momentum 的核心思想是不要只相信当前 batch 的梯度，而是综合过去几步的方向。公式是：

$$v^{k+1} = \beta v^k + g_k,$$

$$w^{k+1} = w^k - \alpha_k v^{k+1}$$

它的作用是：

- 稳定方向被累积；



- 来回震荡的方向被抵消；
- 减少 zig-zag；
- 让训练轨迹更平滑。

Nesterov 动量则进一步加入“提前看一眼”的思想：先按照动量预判一下会走到哪里，再在那里计算梯度进行修正。所以 Momentum / Nesterov 主要改进的是：

随机梯度方向太抖的问题

Momentum 的作用

把多次梯度做平滑平均，稳定的下降方向会被保留，来回摆动的噪声方向会被部分抵消。

直观理解

普通动量像“先看当前坡度再冲”；Nesterov 像“先按惯性预判一下会到哪里，再看那里的坡度”。

自适应步长：解决不同维度尺度不一的问题

AdaGrad / RMSProp：解决不同坐标尺度不一致。有些参数维度梯度很大，有些参数维度梯度很小。如果所有维度都用同一个学习率，就会出现：

- 大梯度维度走太猛；小梯度维度走太慢；
- 稀疏特征很难有效更新。

于是 AdaGrad、RMSProp 引入**坐标级自适应步长**。典型形式是：

$$w_j^{k+1} = w_j^k - \alpha_k \frac{g_{k,j}}{\sqrt{r_{k,j}} + \epsilon}.$$

注意这里是**逐坐标计算**。AdaGrad 用的是**历史平方梯度累积**：

$$r_k = r_{k-1} + g_k \odot g_k.$$

RMSProp 用的是近期平方梯度的指数平均：

$$r_k = \rho r_{k-1} + (1 - \rho)(g_k \odot g_k).$$

所以它们主要改进的是：

不同坐标梯度尺度差异大，一个统一学习率不够用的问题

设当前梯度 $g = (4, 0.4)$ ，基础学习率 $\alpha = 0.1$ 。若历史或近期平方梯度给出

$$r = (16, 0.16), \quad \sqrt{r} = (4, 0.4),$$

这里 $\sqrt{r}$ 是逐坐标开方，因此两个坐标的梯度尺度相差 10 倍。

方法	更新量	更新直觉
普通 SGD	$\Delta w = -0.1(4, 0.4) = (-0.40, -0.04)$	同一个学习率：第 1 坐标走得猛，第 2 坐标走得小。
自适应缩放	$\Delta w \approx -0.1 g / (\sqrt{r} + \epsilon) \approx (-0.10, -0.10)$	大尺度坐标被压小，小尺度或稀疏坐标仍能有效更新。

读法

自适应方法相当于使用坐标级有效学习率 $\alpha_j^{\text{eff}} = \alpha_k / (\sqrt{r_j} + \epsilon)$ 。这里第 1 坐标约为 0.025，第 2 坐标约为 0.25。

Adam: Momentum + RMSProp

Adam 可以理解成把两类思想合在一起：

Adam = 方向平滑 + 坐标自适应步长。

它维护两个量：

一阶矩，用来平滑**梯度方向**：

$$m_k = \beta_1 m_{k-1} + (1 - \beta_1) g_k.$$

二阶矩，用来估计**每个坐标的梯度尺度**：

$$v_k = \beta_2 v_{k-1} + (1 - \beta_2) (g_k \odot g_k).$$

更新时大致是：

$$w_j^{k+1} = w_j^k - \alpha_k \frac{\hat{m}_{k,j}}{\sqrt{\hat{v}_{k,j}} + \epsilon}.$$

所以 Adam 的口语化理解是：用“平滑后的梯度方向”，除以“每个坐标近期梯度尺度”。它同时解决两个问题：

问题	Adam 里的对应机制
梯度方向抖	一阶矩 m_k , 类似 Momentum
不同坐标尺度不同	二阶矩 v_k , 类似 RMSProp

因此 Adam 在深度学习中非常常用。